# Timed IC vs rho on RunPod, std vs fast hasher (2026-11-12)

# Timed IC vs rho on a RunPod CPU pod, std vs fast hasher: the IC is faster up to 2^28 and slower at 2^32

Protocol: `PROTOCOL.md` (frozen in fde73fb405 before data; amendment A1 after a failed first attempt that produced
no measurement). Host: one RunPod Secure Cloud `cpu5c` pod (AMD EPYC 9575F 64-core, 128 threads; the container
may use CPUs 27, 56, 91, 120; every execution pinned to CPU 91), image `runpod/base:1.0.2-ubuntu2404`, pod
yt43r1gavrm8w3 (created, payload uploaded and deleted by `launch.py`; receipts in `launch/`). **3672 executions
(2 builds x 36 configurations x 17 keys x 3 interleaved repeats), 0 errors, every target verified.** Data:
`results/holdout/runs.jsonl`, `opcost.jsonl`, `host.json`, `job.log`, `analysis.txt`, `summary.json`; compact table
`runs.json` (`mk_runs.py`). Every number here is [pcc:unverified] until the claim is signed. Independent validation:
`validation/VALIDATOR_REPORT.md` (pass: both binaries rebuild bit for bit, all 3672 rows replay exactly in their
counters, 301k recovered logs checked with independent EC code, every number recomputes; its amendments are applied
here). The seconds are
diagnostic: a shared cloud host (load average 37.6-93.5 on 128 threads during the run; steal 0), not an isolated lab node.

## Result (solve-phase wall time, fastest IC key / fastest rho key per configuration; median of 4 per cell)

| bits | T | std: IC/rho [95%] | fast: IC/rho [95%] | IC fast/std | rho fast/std | IC/BSGS std / fast |
|---|---|---|---|---|---|---|
| 24 | 1024 | 0.884 [0.861, 0.984] faster | 0.893 [0.809, 0.974] faster | 0.892 | 0.878 | 1.52 / 1.53 |
| 24 | 4096 | 0.733 [0.718, 0.825] faster | 0.776 [0.765, 0.799] faster | 0.890 | 0.845 | 1.47 / 1.46 |
| 24 | 16384 | 0.700 [0.697, 0.712] faster | 0.712 [0.704, 0.726] faster | 0.900 | 0.886 | 1.46 / 1.41 |
| 28 | 1024 | 0.791 [0.766, 0.824] faster | 0.739 [0.719, 0.754] faster | 0.918 | 0.981 | 1.44 / 1.46 |
| 28 | 4096 | 0.833 [0.819, 0.835] faster | 0.779 [0.751, 0.794] faster | 0.926 | 0.989 | 1.48 / 1.50 |
| 28 | 16384 | 0.953 [0.936, 0.995] faster | 0.944 [0.929, 0.975] faster | 0.946 | 0.961 | 1.54 / 1.62 |
| 32 | 1024 | 1.053 [1.029, 1.083] slower | 1.011 [0.996, 1.031] unranked | 0.963 | 1.003 | 1.52 / 1.60 |
| 32 | 4096 | 1.170 [1.136, 1.195] slower | 1.108 [1.076, 1.145] slower | 0.938 | 0.986 | 1.60 / 1.62 |
| 32 | 16384 | 1.179 [1.161, 1.192] slower | 1.148 [1.110, 1.171] slower | 0.949 | 0.975 | 1.58 / 1.60 |

- **Size pattern, both builds.** std: the IC is faster in 12 of 12 configurations at 2^24 (0.70-0.98x) and at 2^28
  (0.77-0.99x) and slower in 12 of 12 at 2^32 (1.03-1.19x). These ranges are per configuration; the table
  gives the cell medians (median of 4 configurations), and each cell decision is a 4-of-4 sign rule (the bootstrap
  interval of a median of 4 equals their range). fast: faster in 12 of 12 at 2^24 and 2^28, slower in
  11 of 12 at 2^32 (1.00-1.17x; 32 b T = 1024 is unranked at 1.011 [0.996, 1.031]).
- **Host replication (std build vs the session VM of `../mt_timed_20261112/`).** 2^24 and 2^32 reproduce in direction
  (VM: 0.78-0.83 and 1.11-1.29). 2^28, unranked or fragile on the VM, is faster in all three cells here
  (0.79-0.95), so the crossover lies between 2^28 and 2^32 on this host.
- **Fast hasher.** It makes both sides faster: the fastest IC by 4-11% (fast/std 0.89-0.96) and the fastest rho by
  0-15% (0.85-1.00), more for rho at 2^24 and more for the IC at 2^28-2^32. It narrows the IC's deficit at 2^32
  (1.05-1.18 to 1.01-1.15) but does not reverse it. The size pattern does not depend on the std hasher. By arm, the
  retention variant ic5rw gains about 15% (fast/std about 0.85), much less than the predicted 0.6.
- **Fixed-key control** (each side's key chosen on repeat 0, scored on repeats 1-2): std faster in the six cells up to
  2^28 and slower in the three at 2^32 (0.69-1.18); fast the same except unranked at 32 b T = 1024 (0.72-1.13).
- **Distance to BSGS:** the IC takes 1.41-1.62x the time of the fastest paired BSGS in every cell of both builds.
- Fastest arms: rho side `rho2xks` at 24 b (all three cells in the fast build, T = 1024 and 4096 in the std build)
  and in 1 of 4 configurations at 28 b T = 1024 (fast), `rhodpu` elsewhere; IC side `ic5` or `ic5w`
  (the retention variants never win in time).

## Controls and their limits

- **A/A is not clean.** The rho side's repeat-0/repeat-1 ratio is 0.97-1.14 (std) and 1.01-1.11 (fast) and its
  interval contains 1 in only 4 of 9 (std) and 3 of 9 (fast) cells: the first round ran about 3% slower than rounds
  2 and 3 (median key time relative to its median: 1.032, 1.000, 1.000), with a drop in host load late in the run.
  Within each round the order of all keys of both builds was a single shuffle, so the drift applies to both sides
  of every comparison; the median |log(repeat/median)| over all keys is 0.008 counting each key's median repeat
  (0.020 without it; 95th percentile 0.11). The IC/rho effects at 2^24 (12-30%) and in most 2^32 cells (5-20%) are
  larger than the drift; the fast build's 2^32, T = 1024 cell (1%) is not, and single-cell magnitudes should be read
  with the drift in mind.
- With n = 4 configurations per cell the bootstrap interval of the median equals the range: each cell decision is a
  4-of-4 sign rule; the size-level counts above (12 of 12) are the strongest evidence.

## Measured prices (`opcost`, std binary, before and after, curve 0 of each size)

One affine addition (1 GAE): 98-140 ns (about 2x faster than the session VM) [pcc:unverified]; I/M 23-27; a canonicalisation plus
one hash of x (two counted native operations): 0.006-0.007 GAE; a std HashMap lookup: 0.06-0.11 GAE at 2^10-2^16
entries, 0.74-1.45 GAE at 2^22-2^24 (relative to the faster field arithmetic, lookups at large tables cost more GAE
here than on the VM). These are microbenchmark loops (independent probes), not the solvers' access patterns.

## Answer (interpretation)

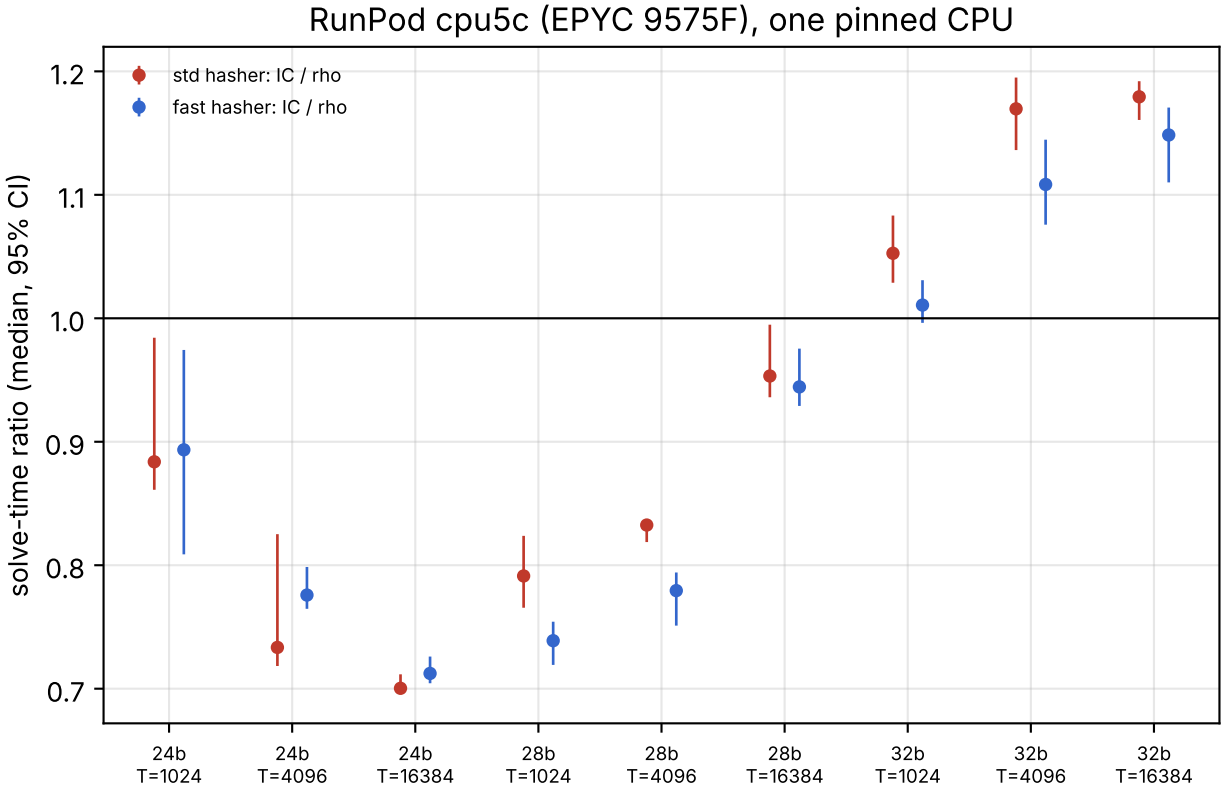
On both hosts and with both hashers, this harness's multi-target index calculus is faster than its multi-target rho
on small groups and slower at 2^32 [pcc:unverified]; on the RunPod host the crossover lies between 2^28 and 2^32 (on the session VM
near 2^28). A faster hash table helps both sides and does not move the crossover past 2^32. The trend runs against
the IC as the group grows, so these measurements give no sign that it beats rho at cryptographic sizes; BSGS remains
faster than the IC everywhere.

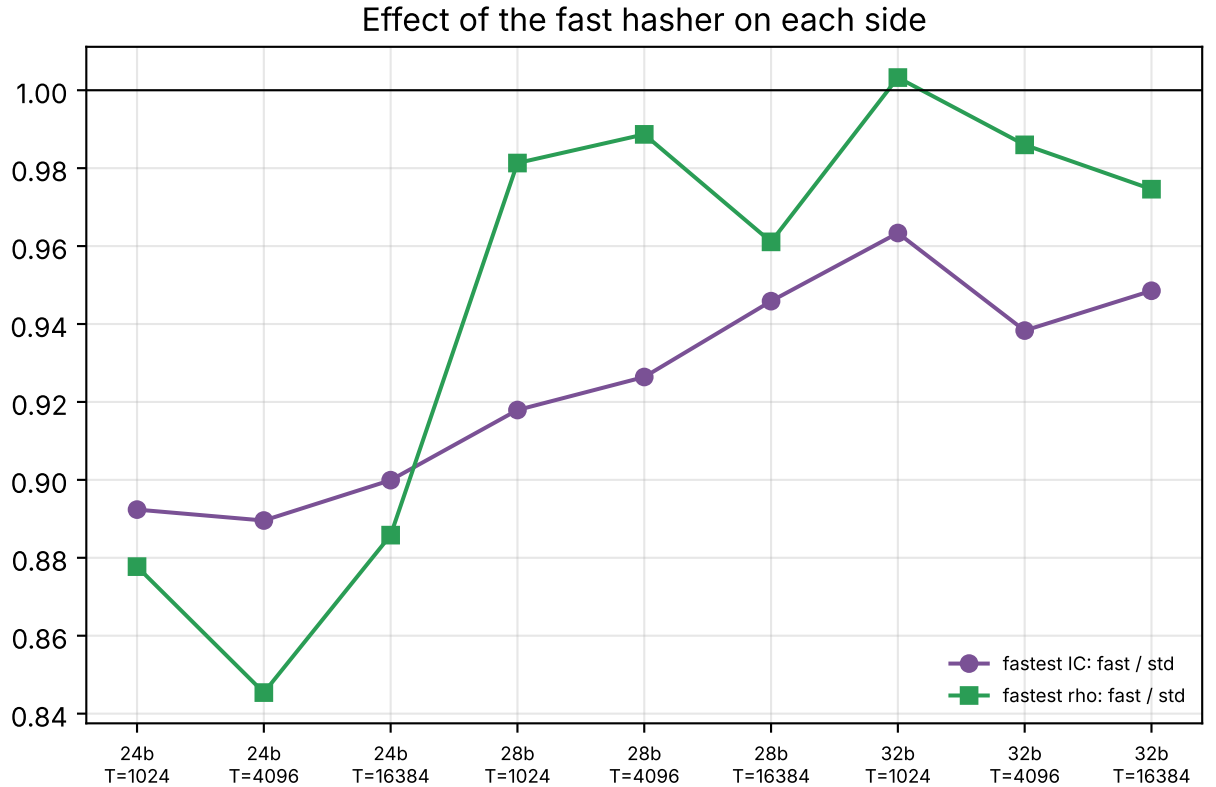
## Pre-registered predictions vs outcome

| prediction | outcome |
|---|---|
| std on RunPod: size pattern reproduces (faster at 2^24, slower at 2^32), magnitudes within about 10% of the VM | direction holds; magnitudes within 10% in 5 of 9 cells |
| fast: hash-heavy arms gain most (ic5rw about 0.6x, ic5ks about 0.8x); ic5 and rhodpu change little; IC/rho falls at every size | partly: ic5rw about 0.85x (not 0.6); th |
| IC slower than BSGS everywhere in both builds | holds (1.41-1.62x) |

## Limits

One shared cloud host (load average 37.6-93.5 of 128 threads; whether the pinned CPU's SMT sibling was idle was not
recorded); the payload's gzip header carries its build time, so `launch.py payload` is reproducible only with that
time (the validator rebuilt it bit for bit); one implementation per build; 4 configurations per cell, the 12 curves and seed of the
earlier timed study (a host replication, not new configurations); A/A drift as above; the counted operations are
identical across builds, so the build comparison isolates the hasher; nothing here is about single-target ECDLP or
about ECC2K-130 / m = 83.


RunPod cpu5c (EPYC 9575F), one pinned CPU
std hasher: IC / rho
fast hasher: IC / rho
solve-time ratio (median, 95% CI)
1.2
1.1
1.0
0.9
0.8
0.7
24b T=1024
24b T=4096
24b T=16384
28b T=1024
28b T=4096
28b T=16384
32b T=1024
32b T=4096
32b T=16384


Effect of the fast hasher on each side
1.00
0.98
0.96
0.94
0.92
0.90
0.88
0.86
0.84
fastest IC: fast / std
fastest rho: fast / std
24b T=1024
24b T=4096
24b T=16384
28b T=1024
28b T=4096
28b T=16384
32b T=1024
32b T=4096
32b T=16384